

1 The problem

- Keyword ATS filters reject real skill over how it's phrased
- Candidates already paid for are lost; can track age/gap bias (Mobley v. Workday)
- Tools mostly rank new applicants; few re-audit past rejections

2 How Relook decides

- LLM turns the JD into keywords (what an ATS scans) vs equivalents (what it misses)
- Per CV: a verdict + verbatim quote for every requirement
- Code, not the model, verifies quotes, scores, and gates on must-haves
- Rescued = ATS rejected it, Relook shortlists it

3 Business value

- Recovers qualified people without lowering the bar (must-haves still gate)
- Audit trail: evidence, rejection-pattern report, CSV export
- Always human-in-the-loop: recommends, never decides

4 Baseline: a simulated ATS

- Our own keyword knock-out: reject if a must-have's keywords are missing
- Not a vendor product: Workday, Greenhouse, Taleo etc. were NOT tested
- Real ATS logic varies (some rank with ML), so real-world lift is unproven
- Stuffed-CV test (n=1): baseline passed a PM; Relook rejected

5 Pros and cons

- + Every verdict cites a verified CV quote; reproducible (temp 0)
- + Works on exports from any ATS; no integration needed
- 16 synthetic CVs only; real CVs are messier, no real ATS data
- Borderline must-haves flip verdicts (94.6% agreement when anonymized)

6 Where it can fail

- A quote proves words exist, not depth of skill or that the claim is true
- Untested: hidden text, non-English or scanned CVs; biased JDs are copied as-is
- Held up in 1 injection + 1 stuffing test; not adversarially tested
- **So it only recommends: a reviewer sees the evidence and decides**

1 The problem

- KT is a meeting and a checklist: nobody tests whether the giver and taker actually agree
- Gaps surface months later as incidents and slow ramp-up
- Project managers can't see KT status, blockers, or who is holding it up

2 How AI does it

- AI asks giver and taker separately; round 2 follows up on weak angles only
- Code verifies quotes and scores readiness against the manager's target
- AI recommends and reviews action items; code decides close / redress / escalate
- Full audit trail: every question, answer, action and decision is logged

3 Business value

- 4/4

planted gaps detected
- 97%

final readiness, up from 57%
- 0/8

false alarms on sound angles
- KT closes on evidence, not on a signed checklist
 - Manager tracker: stage, day n of N, deadline, who each item waits on
 - Turns a KT into an auditable, escalation-ready workflow

1 Brief

PM enters project, roles, stack, dev/test env, duration

2 Q&A

Giver & taker answer independently across 12 KT angles

3 Assess

AI compares both sides; quotes verified in code

4 Actions

Gaps become owned action items, with proof or justification

5 Review

AI reviews responses; weak justifications rejected

6 Decide

Code: Close, one redress round, or Escalate

Redress loop: a rejected item returns to its owner once, with the reviewer's note. Still unresolved: escalated to the manager.

Stack: Python · Streamlit · Google Gemini (structured output), Groq fallback on outage/quota · no GPU needed

Eval: simulated giver/taker personas, 4 planted weak angles, one synthetic project, one run (69s end to end). Target is a stopping rule, not proof of completeness.